МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 4

Студент группы *ИВБО-20-24 Лияскин.В.А*

(подпись)

Ассистент *Шерри.Г.Д*

(подпись)

Отчет представлен «26» сентября 2026 г.

Москва 2026 г.

ЗАДАНИЕ 1. Ранжирующие функции

Задача: Для каждой категории напитков найти 3 самых дорогих товара.

```sql
-- Ранжирование товаров по цене внутри каждой категории
WITH RankedProducts AS (
    SELECT
        p.name AS "Товар",
        p.retail_price AS "Цена",  -- ← исправлено: просто "Цена"
        c.name AS "Категория",
        DENSE_RANK() OVER (
            PARTITION BY p.category_id
            ORDER BY p.retail_price DESC
        ) AS price_rank
    FROM product p
    JOIN category c ON p.category_id = c.id
)
SELECT "Категория", "Товар", "Цена", price_rank AS "Ранг"
FROM RankedProducts
WHERE price_rank <= 3
ORDER BY "Категория", "Ранг";
```

category(+) 1

WITH RankedProducts AS (SELECT p.na | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	Категория	Товар	Цена	Ранг
1	Вода	Вода Святой Источник 0.5л	79,73	1
2	Газированные напитк	Coca-Cola Classic 0.5л	85	1
3	Газированные напитк	Fanta Апельсин 0.5л	82	2
4	Газированные напитк	Sprite 0.5л	82	2
5	Газированные напитк	Pepsi 0.5л	80	3
6	Кофе	Jacobs Кофе в зернах 250г	350	1
7	Молочные коктейли	Молочный коктейль Ваниль	95	1
8	Соки и нектары	Rich Апельсиновый 1л	120	1
9	Соки и нектары	Добрый Яблочный 1л	115	2
10	Чай	Lipton Ice Tea 0.5л	75	1
11	Энергетики	Red Bull 0.25л	150	1
12	Энергетики	Monster Energy 0.5л	145	2

1. **PARTITION BY p.category_id** — делит все товары на независимые группы (окна) по категориям. В каждом окне ранжирование начинается заново с 1.
2. **ORDER BY p.retail_price DESC** — внутри каждого окна товары сортируются по убыванию цены (самые дорогие получают ранг 1).
3. **DENSE_RANK()** vs RANK() vs ROW_NUMBER():
    - ROW_NUMBER() — всегда уникальные номера (1, 2, 3, 4)

- RANK() — одинаковые значения = одинаковый ранг, но следующий ранг **пропускается** (1, 2, 2, 4)
- DENSE_RANK() — одинаковые значения = одинаковый ранг, **без пропусков** (1, 2, 2, 3) ← используем его

ЗАДАНИЕ 2. Агрегатные оконные функции (нарастающий итог)

Задача: Рассчитать кумулятивную (нарастающую) сумму поставок по месяцам для каждого поставщика.

```sql
-- Нарастающий итог поставок по месяцам
WITH MonthlySupply AS (
    -- Шаг 1: агрегируем поставки по месяцам и поставщикам
    SELECT
        sup.name AS "Поставщик",
        DATE_TRUNC('month', s.date)::DATE AS "Месяц",
        SUM(s.summa) AS "Сумма за месяц"
    FROM supply s
    JOIN supplier sup ON s.supplier_id = sup.id
    GROUP BY sup.name, DATE_TRUNC('month', s.date)
)
-- Шаг 2: считаем нарастающий итог
SELECT
    "Поставщик",
    "Месяц",
    "Сумма за месяц",
    SUM("Сумма за месяц") OVER (
        PARTITION BY "Поставщик"    -- для каждого поставщика отдельно
        ORDER BY "Месяц"            -- в хронологическом порядке
    ) AS "Нарастающий итог"
FROM MonthlySupply
ORDER BY "Поставщик", "Месяц";
```

supplier 1

WITH MonthlySupply AS (SELECT sup.n | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	Поставщик	Месяц	Сумма за месяц	Нарастающий итог
1	АО Вода Чистая	2024-03-01	19 000	19 000
2	АО Вода Чистая	2024-05-01	25 000	44 000
3	ЗАО АлкоТрейд	2024-01-01	32 000	32 000
4	ЗАО АлкоТрейд	2024-04-01	42 000	74 000
5	ИП Иванов П.С.	2024-02-01	28 000	28 000
6	ИП Иванов П.С.	2024-04-01	35 000	63 000
7	ООО Напитки Плюс	2024-01-01	45 000	45 000
8	ООО Напитки Плюс	2024-03-01	38 000	83 000
9	ООО Соки России	2024-02-01	51 000	51 000
10	ООО Соки России	2024-05-01	47 000	98 000

Ключевое отличие от GROUP BY:

GROUP BY → 1 строка на группу (теряем детализацию)

SUM() OVER (PARTITION BY ... ORDER BY ...) → все строки сохраняются, но добавляется столбец с кумулятивной суммой

Как работает нарастающий итог:

Когда в OVER() есть ORDER BY, окно по умолчанию = все строки от начала раздела до текущей

Для января: сумма = только январь

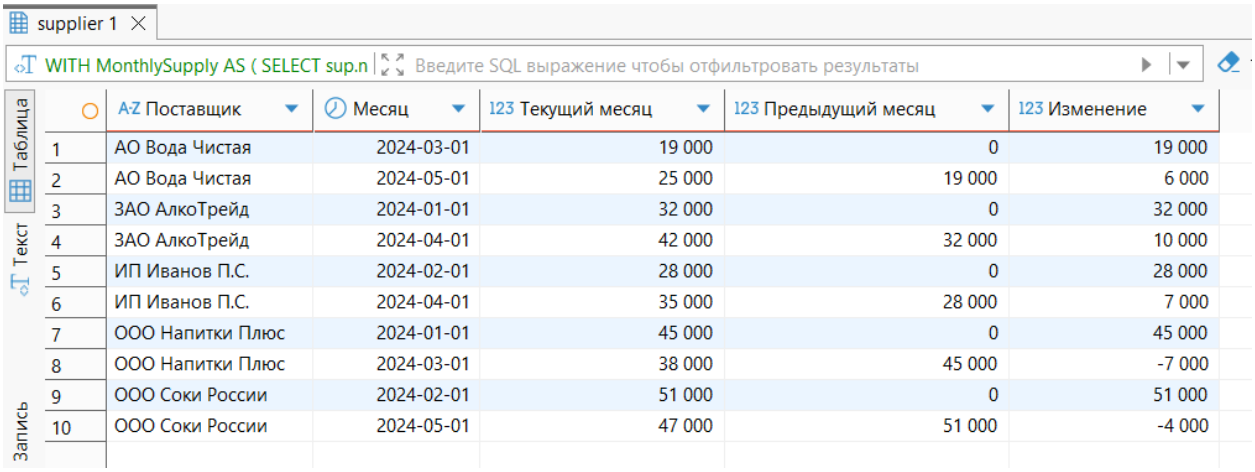
Для февраля: сумма = январь + февраль

Для марта: сумма = январь + февраль + март

ЗАДАНИЕ 3. Функции смещения (LAG)

Задача: Сравнить сумму поставок текущего месяца с предыдущим для каждого поставщика.

```sql
-- Сравнение поставок текущего и предыдущего месяца
WITH MonthlySupply AS (
    SELECT
        sup.name AS "Поставщик",
        DATE_TRUNC('month', s.date)::DATE AS "Месяц",
        SUM(s.summa) AS "Сумма"
    FROM supply s
    JOIN supplier sup ON s.supplier_id = sup.id
    GROUP BY sup.name, DATE_TRUNC('month', s.date)
)
SELECT
    "Поставщик",
    "Месяц",
    "Сумма" AS "Текущий месяц",
    -- LAG: берём значение из предыдущей строки (смещение = 1)
    LAG("Сумма", 1, 0) OVER (
        PARTITION BY "Поставщик"
        ORDER BY "Месяц"
    ) AS "Предыдущий месяц",
    -- Разница между месяцами
    "Сумма" - LAG("Сумма", 1, 0) OVER (
        PARTITION BY "Поставщик"
        ORDER BY "Месяц"
    ) AS "Изменение"
FROM MonthlySupply
ORDER BY "Поставщик", "Месяц";
```



supplier 1

WITH MonthlySupply AS (SELECT sup.n | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	Поставщик	Месяц	Текущий месяц	Предыдущий месяц	Изменение
1	АО Вода Чистая	2024-03-01	19 000	0	19 000
2	АО Вода Чистая	2024-05-01	25 000	19 000	6 000
3	ЗАО АлкоТрейд	2024-01-01	32 000	0	32 000
4	ЗАО АлкоТрейд	2024-04-01	42 000	32 000	10 000
5	ИП Иванов П.С.	2024-02-01	28 000	0	28 000
6	ИП Иванов П.С.	2024-04-01	35 000	28 000	7 000
7	ООО Напитки Плюс	2024-01-01	45 000	0	45 000
8	ООО Напитки Плюс	2024-03-01	38 000	45 000	-7 000
9	ООО Соки России	2024-02-01	51 000	0	51 000
10	ООО Соки России	2024-05-01	47 000	51 000	-4 000

LAG(column, offset, default) — функция смещения назад:

column — из какого столбца брать значение

offset — на сколько строк назад (по умолчанию 1)

default — что вернуть, если предыдущей строки нет (для первой строки в разделе)

LEAD(column, offset, default) — зеркальная функция, смещение вперёд (для прогноза).

Почему LAG, а не self-JOIN?

Self-JOIN требует сложного соединения таблицы с самой собой

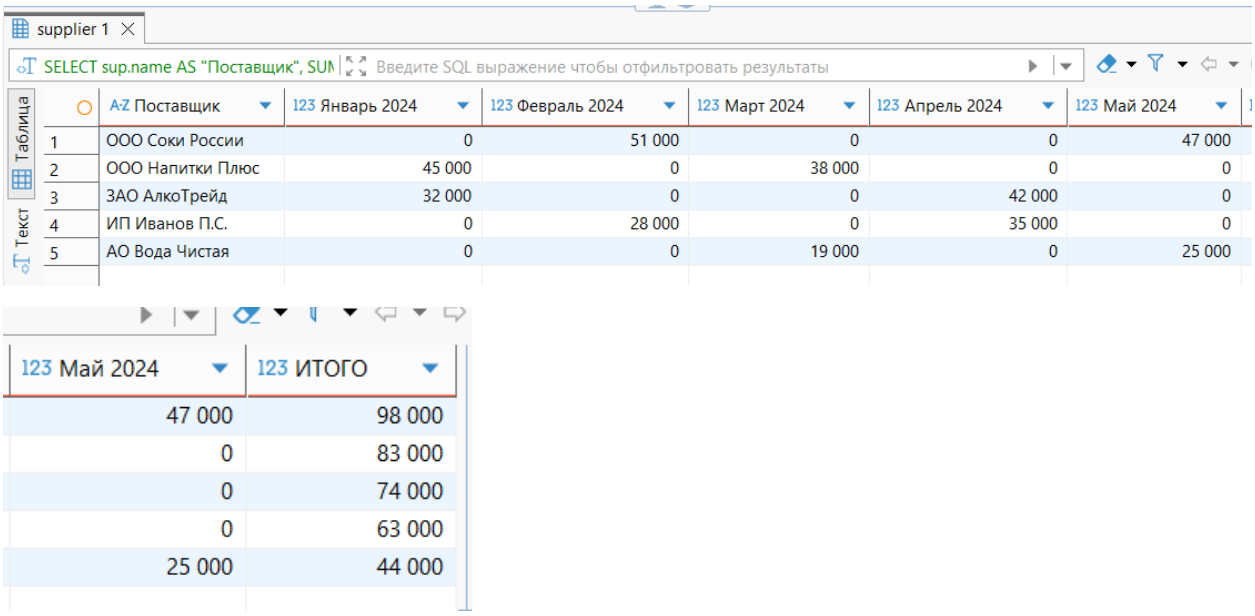
LAG делает то же самое в одну строку кода

LAG работает быстрее и читаемее

ЗАДАНИЕ 4. Построение сводной таблицы (PIVOT)

Задача: Показать сумму поставок каждого поставщика по месяцам в виде сводной таблицы (поставщики в строках, месяцы в столбцах).

```sql
-- Сводная таблица: поставщики × месяцы
SELECT
    sup.name AS "Поставщик",
    SUM(CASE WHEN DATE_TRUNC('month', s.date)::DATE = '2024-01-01'
             THEN s.summa ELSE 0 END) AS "Январь 2024",
    SUM(CASE WHEN DATE_TRUNC('month', s.date)::DATE = '2024-02-01'
             THEN s.summa ELSE 0 END) AS "Февраль 2024",
    SUM(CASE WHEN DATE_TRUNC('month', s.date)::DATE = '2024-03-01'
             THEN s.summa ELSE 0 END) AS "Март 2024",
    SUM(CASE WHEN DATE_TRUNC('month', s.date)::DATE = '2024-04-01'
             THEN s.summa ELSE 0 END) AS "Апрель 2024",
    SUM(CASE WHEN DATE_TRUNC('month', s.date)::DATE = '2024-05-01'
             THEN s.summa ELSE 0 END) AS "Май 2024",
    SUM(s.summa) AS "ИТОГО"
FROM supply s
JOIN supplier sup ON s.supplier_id = sup.id
GROUP BY sup.name
ORDER BY "ИТОГО" DESC;
```



supplier 1

SELECT sup.name AS "Поставщик", SUM | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	Поставщик	Январь 2024	Февраль 2024	Март 2024	Апрель 2024	Май 2024
1	ООО Соки России	0	51 000	0	0	47 000
2	ООО Напитки Плюс	45 000	0	38 000	0	0
3	ЗАО АлкоТрейд	32 000	0	0	42 000	0
4	ИП Иванов П.С.	0	28 000	0	35 000	0
5	АО Вода Чистая	0	0	19 000	0	25 000

Май 2024	ИТОГО
47 000	98 000
0	83 000
0	74 000
0	63 000
25 000	44 000

PIVOT

Это преобразование данных из «длинного» формата в «широкий»:

Длинный: (Поставщик, Месяц, Сумма) — много строк

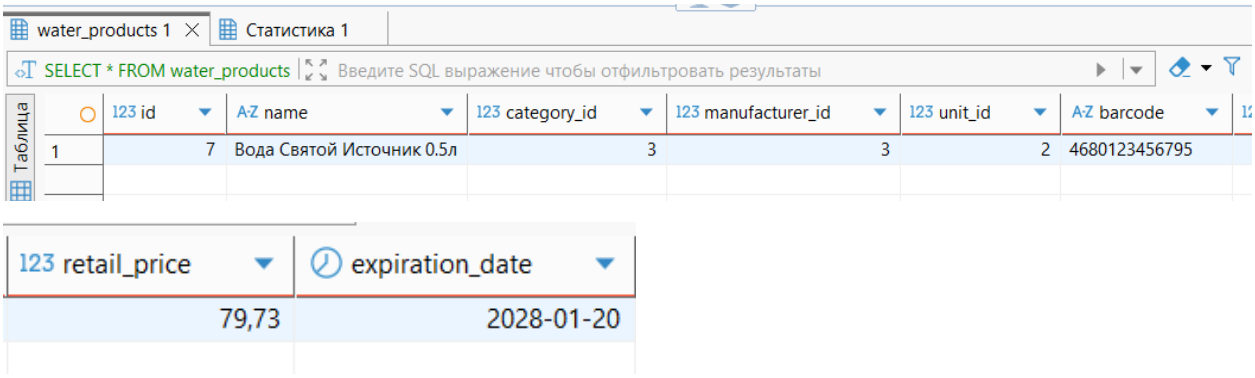
Широкий: (Поставщик, Янв, Фев, Мар, ...) — одна строка на поставщика

ЗАДАНИЕ 5. Модифицируемое представление

Задача: Создать представление, показывающее только товары категории «Вода».

```sql
-- Создаём модифицируемое представление
CREATE OR REPLACE VIEW water_products AS
SELECT
    id,
    name,
    category_id,
    manufacturer_id,
    unit_id,
    barcode,
    retail_price,
    expiration_date
FROM product
WHERE category_id = (SELECT id FROM category WHERE name = 'Вода');

-- Проверяем представление
SELECT * FROM water_products;
```



Почему это представление модифицируемое?

PostgreSQL автоматически разрешает DML-операции через представление, если:

В FROM только одна базовая таблица (product)

Нет GROUP BY, HAVING, DISTINCT, LIMIT

Нет агрегатных/оконных функций

Нет UNION/INTERSECT/EXCEPT

Нет вычисляемых полей в SELECT (только прямые столбцы)

ЗАДАНИЕ 6. Модификация данных через представление

Задача: Добавить и удалить товар через представление water_products.

```sql
-- 6.1. Добавление товара через представление
INSERT INTO water_products
    (name, category_id, manufacturer_id, unit_id, barcode, retail_price, expiration_date)
VALUES
    ('Боржоми 0.5л',
     (SELECT id FROM category WHERE name = 'Вода'),
     3,  -- Nestlé
     2,  -- мл
     '4680123456801',
     120.00,
     '2027-12-31');

-- Проверяем, что запись появилась в основной таблице product
SELECT * FROM product WHERE name = 'Боржоми 0.5л';

-- 6.2. Удаление товара через представление
DELETE FROM water_products
WHERE name = 'Боржоми 0.5л';

-- Проверяем, что запись удалена из основной таблицы
SELECT * FROM product WHERE name = 'Боржоми 0.5л';  -- должно быть пусто
```

product 1 | product 1 (2) | Статистика 1

Таблица

	id	name	category_id	manufacturer_id	unit_id	barcode
1	15	Боржоми 0.5л	3	3	2	4680123456801

retail_price	expiration_date
120	2027-12-31

product 1 | product 1 (2) | Статистика 1

Таблица

id	name	category_id	manufacturer_id	unit_id	barcode	retail_price

ЗАДАНИЕ 7. Немодифицируемое аналитическое представление

Задача: Создать представление со статистикой по производителям (количество товаров, средняя цена, количество истекающих товаров).

```sql
-- Аналитическое представление (немодифицируемое)
CREATE OR REPLACE VIEW manufacturer_summary AS
SELECT
    m.name AS "Производитель",
    c.name AS "Страна",
    COUNT(p.id) AS "Кол-во товаров",
    ROUND(AVG(p.retail_price), 2) AS "Средняя цена",
    MIN(p.retail_price) AS "Мин. цена",
    MAX(p.retail_price) AS "Макс. цена",
    -- Количество товаров с истекающим сроком (в ближайшие 6 месяцев)
    COUNT(p.id) FILTER (
        WHERE p.expiration_date IS NOT NULL
        AND p.expiration_date <= CURRENT_DATE + INTERVAL '6 months'
    ) AS "Истекает в течение 6 мес."
FROM manufacturer m
JOIN country c ON m.country_id = c.id
LEFT JOIN product p ON m.id = p.manufacturer_id
GROUP BY m.id, m.name, c.name;

-- Проверяем представление
SELECT * FROM manufacturer_summary ORDER BY "Кол-во товаров" DESC;
```

manufacturer_summary 1 | Статистика 1

Таблица | Текст

	Производитель	Страна	Кол-во товаров	Средняя цена	Мин. цена	123
1	Coca-Cola Company	США	4	91	82	
2	Monster Energy	США	1	145	145	
3	Nestlé	Франция	1	79,73	79,73	
4	Черноголовка	Россия	1	120	120	
5	Балтмолпрод	Россия	1	95	95	
6	Jacobs	Германия	1	350	350	
7	Red Bull	Германия	1	150	150	
8	Lipton	Великобритания	1	75	75	
9	PepsiCo	США	1	80	80	
10	Danone	Франция	0	[NULL]	[NULL]	
11	Wimm-Bill-Dann	Россия	0	[NULL]	[NULL]	
12	Очаково	Россия	0	[NULL]	[NULL]	

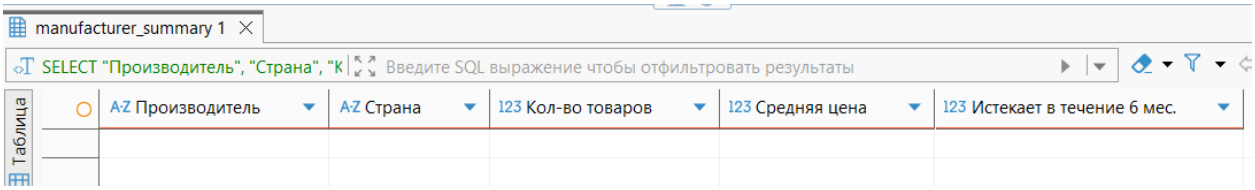
ультаты

Макс. цена	Истекает в течение 6 мес.
115	1
145	0
79,73	0
120	1
95	1
350	0
150	0
75	0
80	0
[NULL]	0
[NULL]	0
[NULL]	0

ЗАДАНИЕ 8. Использование аналитического представления

Задача: Найти производителей со средней ценой выше 100 рублей и более чем 1 товаром.

```sql
-- Используем представление как обычную таблицу
SELECT
    "Производитель",
    "Страна",
    "Кол-во товаров",
    "Средняя цена",
    "Истекает в течение 6 мес."
FROM manufacturer_summary
WHERE "Средняя цена" > 100
  AND "Кол-во товаров" > 1
ORDER BY "Средняя цена" DESC;
```



Сложная логика (JOIN + GROUP BY + агрегаты + FILTER) спрятана внутри представления. Пользователь видит простую «таблицу» и может фильтровать/сортировать её как угодно.

ЗАДАНИЕ 9. Материализованное представление

Задача: Создать материализованное представление для отчёта по продажам и продемонстрировать обновление.

```sql
-- Используем представление как обычную таблицу
-- 9.1. Создаём материализованное представление
CREATE MATERIALIZED VIEW sales_by_client AS
SELECT
    c.id AS client_id,
    COALESCE((c.full_name).last_name || ' ' || (c.full_name).first_name, 'Аноним') AS "Клиент",
    COUNT(s.id) AS "Кол-во покупок",
    SUM(s.total_amount) AS "Общая сумма покупок",
    ROUND(AVG(s.total_amount), 2) AS "Средний чек"
FROM client c
LEFT JOIN sale s ON c.id = s.client_id
GROUP BY c.id, c.full_name
ORDER BY "Общая сумма покупок" DESC;

-- 9.2. Проверяем данные
SELECT * FROM sales_by_client;

-- 9.3. Добавляем новую продажу (данные в базовой таблице меняются)
INSERT INTO sale(number, date, employee_id, client_id, payment_type_id, total_amount)
VALUES ('SALE-2024-0013', '2024-01-21 10:00:00+03', 1, 1, 2, 500.00);

-- 9.4. Проверяем материализованное представление — данные НЕ изменились!
SELECT * FROM sales_by_client WHERE client_id = 1;
-- "Кол-во покупок" и "Общая сумма" — старые, без новой продажи

-- 9.5. Обновляем материализованное представление
REFRESH MATERIALIZED VIEW sales_by_client;

-- 9.6. Проверяем снова — теперь данные актуальны!
SELECT * FROM sales_by_client WHERE client_id = 1;
-- "Кол-во покупок" увеличилось на 1, "Общая сумма" +500 руб.
```

sales_by_client 1 | sales_by_client 1 (2) | sales_by_client 1 (3) | Статистика 1

SELECT * FROM sales_by_client | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст | Запись

	client_id	Клиент	Кол-во покупок	Общая сумма покупок	Средний чек
1	11	Аноним	0	[NULL]	[NULL]
2	6	Морозова Елена	1	1 250	1 250
3	9	Соколов Андрей	1	920	920
4	4	Козлова Анна	1	890	890
5	8	Лебедева Ольга	1	780	780
6	5	Новиков Игорь	1	670	670
7	3	Сидоров Дмитрий	1	580	580
8	10	Павлова Татьяна	1	450	450
9	1	Иванов Алексей	1	450	450
10	7	Волков Сергей	1	340	340
11	2	Петрова Мария	1	320	320

sales_by_client 1 | sales_by_client 1 (2) | sales_by_client 1 (3) | Статистика 1

SELECT * FROM sales_by_client WHERE | Введите SQL выражение чтобы отфильтровать результаты

Таблица

	client_id	Клиент	Кол-во покупок	Общая сумма покупок	Средний чек
1	1	Иванов Алексей	1	450	450

sales_by_client 1 | sales_by_client 1 (2) | sales_by_client 1 (3) | Статистика 1

SELECT * FROM sales_by_client WHERE | Введите SQL выражение чтобы отфильтровать результаты

Таблица

	client_id	Клиент	Кол-во покупок	Общая сумма покупок	Средний чек
1	1	Иванов Алексей	2	950	475

Материализованное представление — это снимок данных на момент создания. PostgreSQL не отслеживает изменения в базовых таблицах. Нужно явно вызвать REFRESH MATERIALIZED VIEW.